

Multi-Task Vision System

using YOLO (v8 and Beyond)

Student Name	PRN
Parimal Ahire	202301040067
Atharva Suryawanshi	202301040283
Rajveersinh Kher	202301040233
Mohit Patil	202301040272

Academic Year 2025 – 2026

1. Objective

The goal of this lab assignment is straightforward but ambitious — build a single, unified computer vision system capable of performing four entirely different tasks, all using the YOLO (You Only Look Once) family of models. Rather than training separate, unrelated models, we wanted to experience firsthand how a common framework can be adapted across detection, classification, pose estimation, and oriented bounding box tasks.

We chose the Ultralytics YOLOv8/v11 framework because it provides a clean, consistent API regardless of which vision task you are working on. Everything from data loading to model export follows the same pattern, which made it much easier to stay organized across four different experiments.

What this system does:

- Object Detection — identifying and localising objects with axis-aligned bounding boxes and confidence scores
- Image Classification — predicting which category an image belongs to (e.g., which animal species)
- Pose Estimation — detecting 17 human body keypoints and rendering a connected skeleton overlay
- Oriented Bounding Box (OBB) Detection — detecting objects at arbitrary rotations using angled bounding boxes
- Local Deployment — running all four models on a personal machine with no cloud dependency

Why multiple tasks?

Real-world computer vision systems rarely do just one thing. A smart surveillance camera detects people AND estimates their pose. An industrial inspection robot detects components AND classifies defects. Building all four tasks in a single codebase gives us a much deeper appreciation of what modern YOLO architectures can do.

2. Course & Assignment Details

Before getting into the technical work, here is a quick summary of the course context and assignment parameters that guided our decisions throughout this project.

Course Name	Deep Learning
Lab Assignment	Lab Assignment 3
Semester	VI
Academic Year	2025 – 2026
Topic	Multi-Task Vision System using YOLO (v8 and Beyond)
Framework Used	Ultralytics YOLOv8 / YOLOv11 (Nano models)
Training Mode	CPU-only — no GPU used
Dataset Source	Roboflow Universe (all four datasets)

2.1 Group Members

Sr.	Student Name	PRN	Contribution Area
1	Parimal Ahire	202301040067	Object Detection + Deployment
2	Atharva Suryawanshi	202301040283	Image Classification + Report
3	Rajveersinh Kher	202301040233	Pose Estimation + Dataset Prep
4	Mohit Patil	202301040272	OBB Detection + Integration

3. Tools and Technologies

We kept our technology stack lean and focused. Everything was chosen with two goals in mind: make it work on a standard laptop without a GPU, and keep the codebase clean enough that any group member could pick up any module and understand it quickly.

3.1 Software Stack

Category	Tool / Library	Version / Notes
Programming Language	Python	3.11 — latest stable with full YOLO support
YOLO Framework	Ultralytics YOLO	v8 / v11 — unified API for all tasks
Deep Learning Backend	PyTorch	2.0+ — CPU inference supported
Image Processing	OpenCV (cv2)	4.7+ — reading, resizing, colour conversion
Image I/O	Pillow (PIL)	9.5+ — format conversion, display
Visualisation	Matplotlib	3.7+ — training graphs, result plots
Numerical Computing	NumPy	1.24+ — array ops, preprocessing
Dataset Platform	Roboflow	Exported in YOLOv8 format
IDE	Visual Studio Code	With Python and Pylance extensions

3.2 Hardware Configuration

Processor	Intel Core i5-1335U (13th Gen, 10 cores)
GPU	Not used — all training and inference ran on CPU
RAM	16 GB DDR4
Storage	SSD (fast model checkpoint read/write)
OS	Windows 11 (64-bit)
Python	3.11 (via Anaconda environment)

4. Project Structure

We designed the folder structure with modularity in mind — each task lives in its own directory and can be trained, tested, and deployed independently. Anyone reading the project for the first time should be able to navigate to any task without confusion.

Lab-Assignment-3-YOLO-Model/	
├── detection/	← Object detection (train.py, predict.py)
├── classification/	← Image classification (train.py, predict.py)
├── pose/	← Pose estimation (train.py, predict.py)
├── obb/	← OBB detection (train.py, predict.py)
├── datasets/	
│ ├── detection/	← Helmet Detection dataset (YOLO format)
│ ├── classification/	← Animal Classification dataset
│ ├── pose/	← Human Pose dataset (COCO keypoints)
│ └── obb/	← PCB OBB dataset (DOTA format)
├── models/	← Saved .pt weight files (best + last)
├── test_images/	← Unseen images for inference testing
├── runs/	← Auto-generated: training logs + checkpoints
├── results/	← Auto-saved annotated prediction outputs
├── deploy.py	← Interactive local deployment script
├── requirements.txt	← All Python dependencies
├── README.md	← Setup guide + usage instructions
└── report.pdf	← This report

Every task folder contains its own train.py and predict.py script. This separation means you can run a single task without affecting the others, and each script is self-contained — it loads its own dataset path, model configuration, and output directory.

5. Dataset Details

All four datasets were downloaded from Roboflow Universe. Roboflow was a natural choice because it handles the messy parts of dataset preparation — format conversion, train/val/test splits, and augmentation previews — so we could focus on the actual model training.

Task	Dataset Name	Classes	Roboflow Link
Object Detection	Helmet Detection	2 (helmet, no-helmet)	universe.roboflow.com/humanhead-azp04/helmet-detection-x2yqr
Image Classification	Animal Classification	Multiple animal species	universe.roboflow.com/detection-aq67m/animal-classification-bckhx
Pose Estimation	Human Pose Dataset	1 person + 17 keypoints	universe.roboflow.com/custom-wyfc3/human-pose-3vmzj
OBB Detection	PCB Component Detection	PCB component types	universe.roboflow.com/hieu2959/obb-b-bkiy

5.1 Export Format

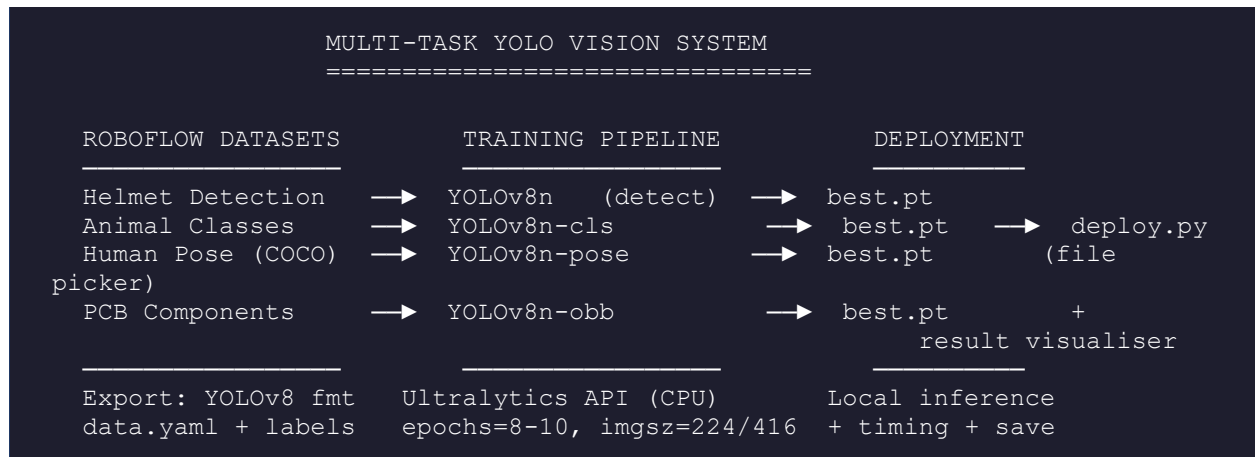
Each dataset was exported from Roboflow using the YOLOv8 export option. This generates a data.yaml file along with images/ and labels/ directories for train, valid, and test splits. The YAML file contains the class names and dataset paths — exactly what the Ultralytics trainer expects.

5.2 Preprocessing Steps

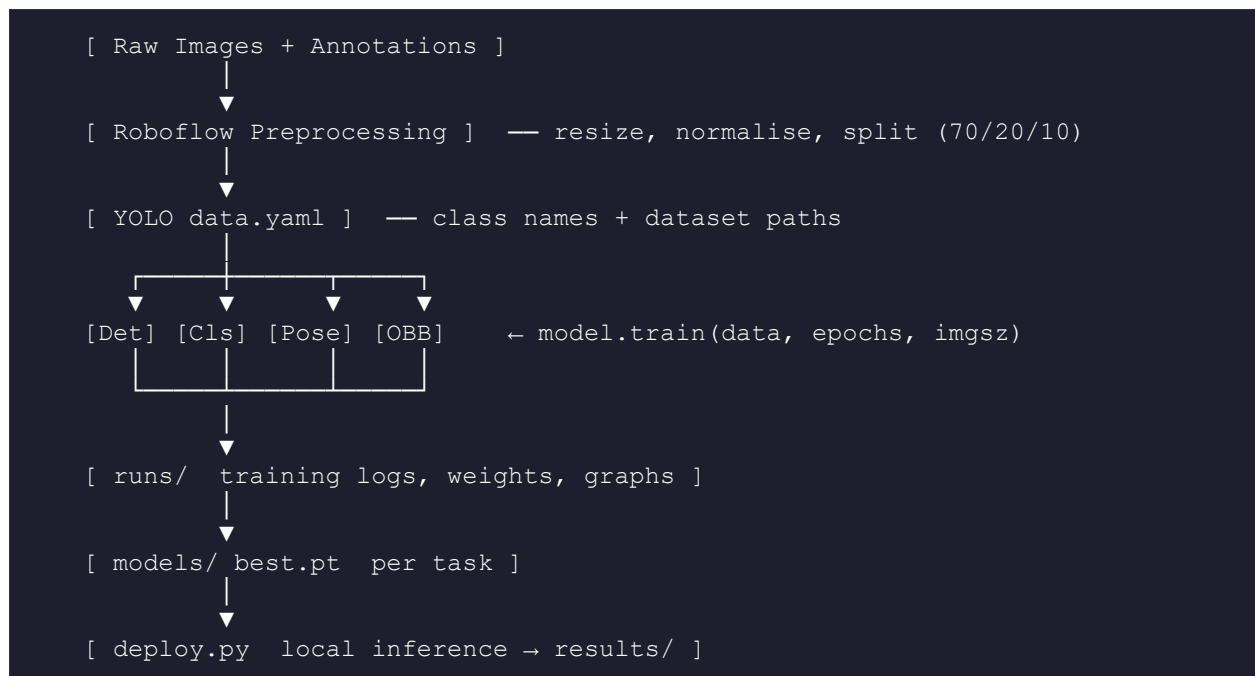
- Image resizing to match each model's input size (416x416 for detection/pose/OBB, 224x224 for classification)
- Pixel normalisation to [0.0, 1.0] range — handled automatically by the Ultralytics dataloader
- YOLO annotation format verification — each label file checked for valid class ID and bounding box coordinates
- Train / validation / test split confirmation (Roboflow default: 70% / 20% / 10%)
- Augmentation: horizontal flip, scale jitter, mosaic (enabled by default in Ultralytics pipeline)

6. System Architecture

Before diving into each task individually, it helps to understand how all four parts fit together. The diagram below shows the overall pipeline — from raw data through to the deployment interface.



6.1 Data Flow



7. Implementation

This is the core of the assignment. Each of the four parts is described below with its training parameters, the reasoning behind our choices, code snippets, and what the output looks like. We ran everything on CPU so training was slower than it would be on a GPU, but all four models converged successfully within 8 to 10 epochs.

Part A — Object Detection

Part A — Object Detection	
Model	YOLOv8n (Nano) — detection variant
Dataset	Helmet Detection (Roboflow)
Classes	2 — helmet / no-helmet
Epochs	10
Image Size	416 x 416 pixels
Task	Bounding box detection with class labels + confidence scores
Output	Annotated images saved to runs/detect/

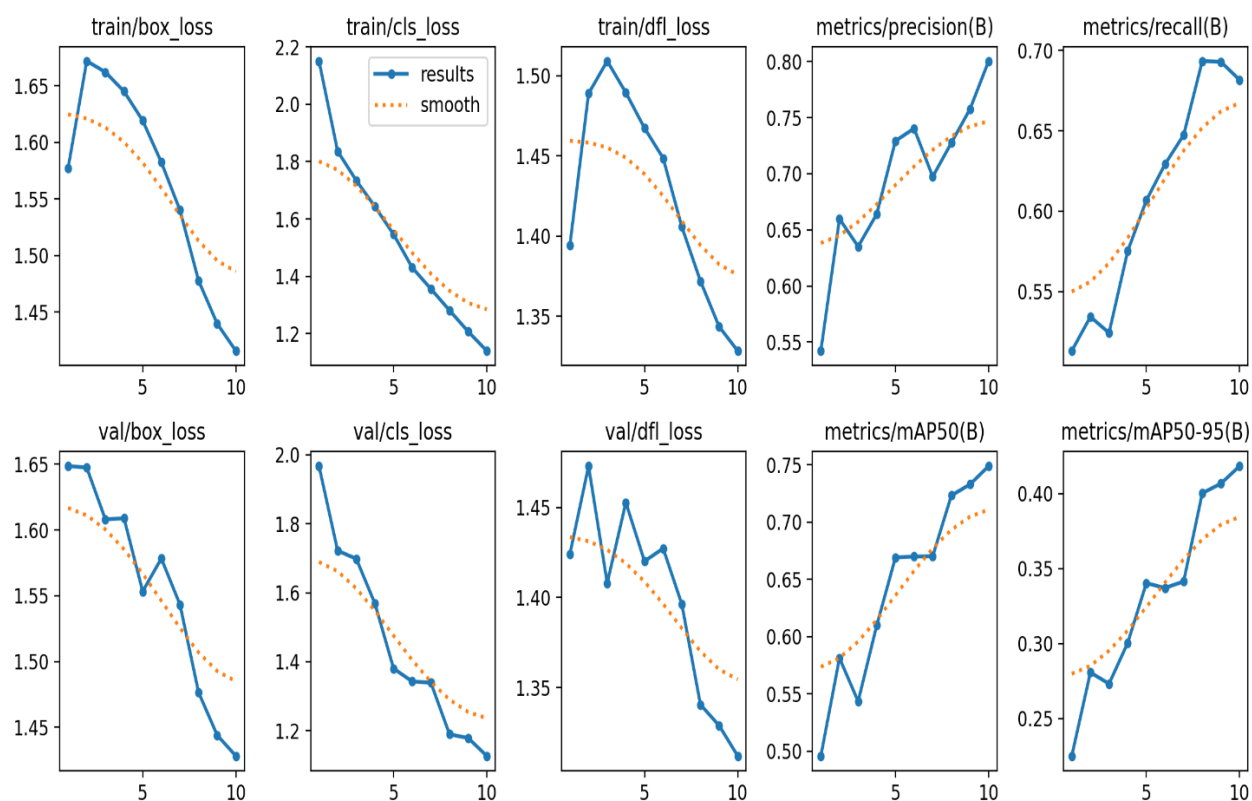
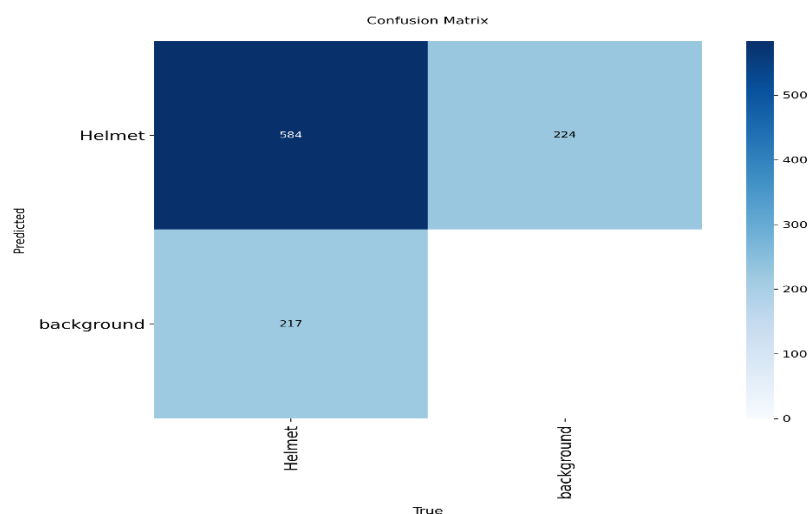
Object detection was the most straightforward of the four tasks. We used the helmet detection dataset because it has a clear practical use case (construction site safety monitoring) and the two-class setup makes it easy to evaluate whether the model is working correctly.

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
results = model.train(
    data='datasets/detection/data.yaml',
    epochs=10, imgsz=416,
    project='runs/detect', name='helmet_v1')
model.predict('test_images/detection/', save=True)
```

Expected Output

- Bounding boxes drawn around helmets and heads
- Class label (helmet or no-helmet) displayed above each box
- Confidence score shown alongside each label (e.g., 0.87)
- Training curves (box loss, cls loss, mAP50) saved to runs/detect/

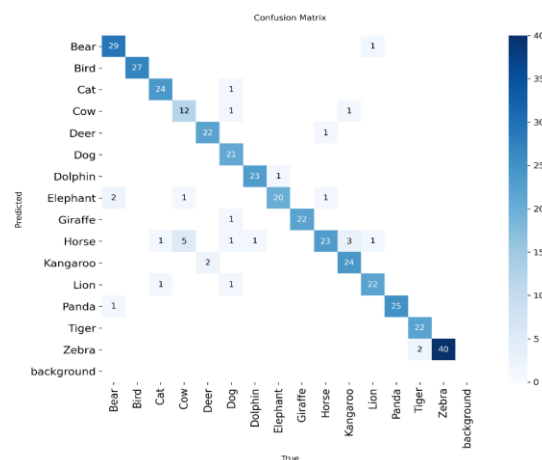


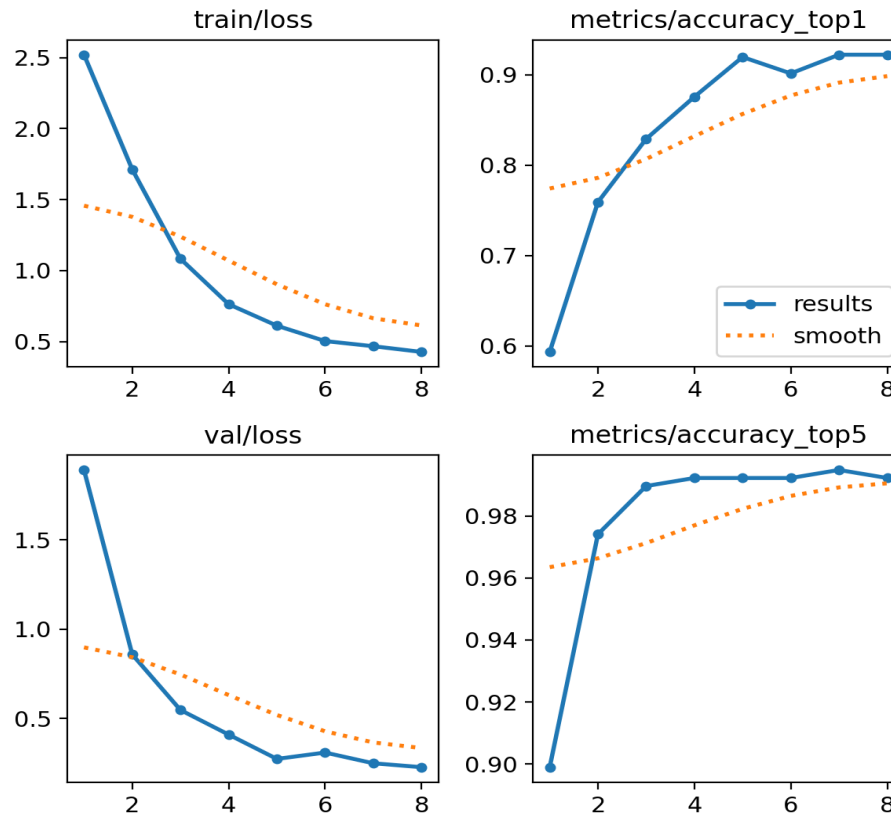
Part B — Image Classification

Part B — Image Classification	
Model	YOLOv8n-cls — classification variant
Dataset	Animal Classification (Roboflow)
Epochs	8
Image Size	224 x 224 pixels (standard for classification)
Task	Multi-class image classification
Output	Predicted class + top-5 probability scores

Classification is a different problem from detection — instead of finding where objects are, the model only needs to say what the whole image contains. The YOLOv8-cls variant replaces the detection head with a classification head and is fine-tuned from ImageNet pretrained weights. We used 224x224 as the input resolution because this is the standard size that most ImageNet-pretrained models expect.

```
model = YOLO('yolov8n-cls.pt')
model.train(
    data='datasets/classification/',
    epochs=8, imgsz=224,
    project='runs/classify', name='animals_v1')
model.predict('test_images/classification/', save=True)
```





Part C — Pose Estimation

Part C — Pose Estimation	
Model	YOLOv8n-pose — pose estimation variant
Dataset	Human Pose Dataset (Roboflow / COCO keypoint format)
Keypoints	17 COCO keypoints per person
Epochs	10
Image Size	416 x 416 pixels
Output	Keypoint coordinates + connected skeleton overlay

Pose estimation is the most visually striking of the four tasks. The model has two jobs simultaneously: first, detect each person in the frame; second, for each detected person, predict

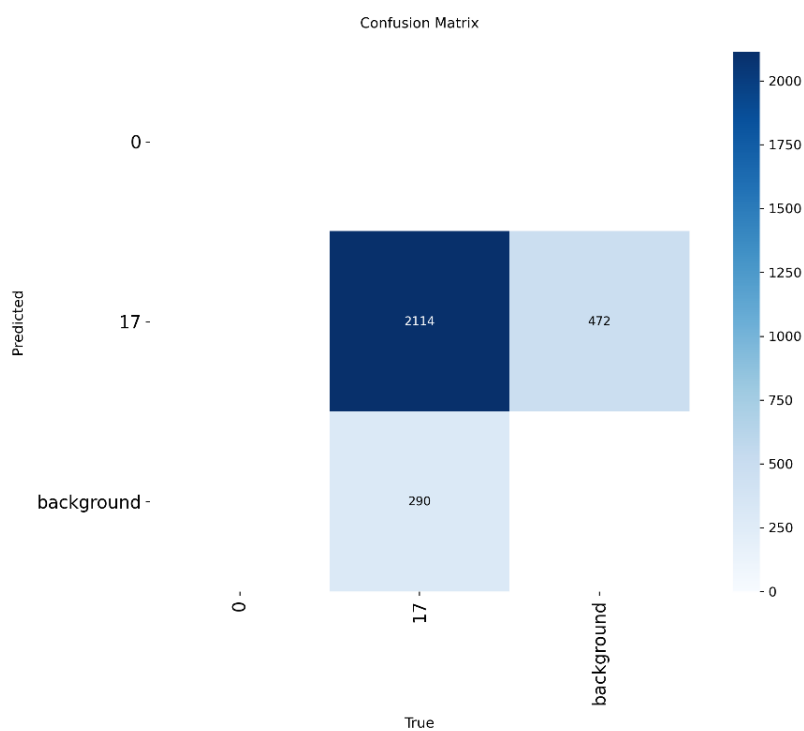
the locations of 17 body keypoints. The keypoints are then connected according to the COCO skeleton definition to produce the familiar stick-figure overlay.

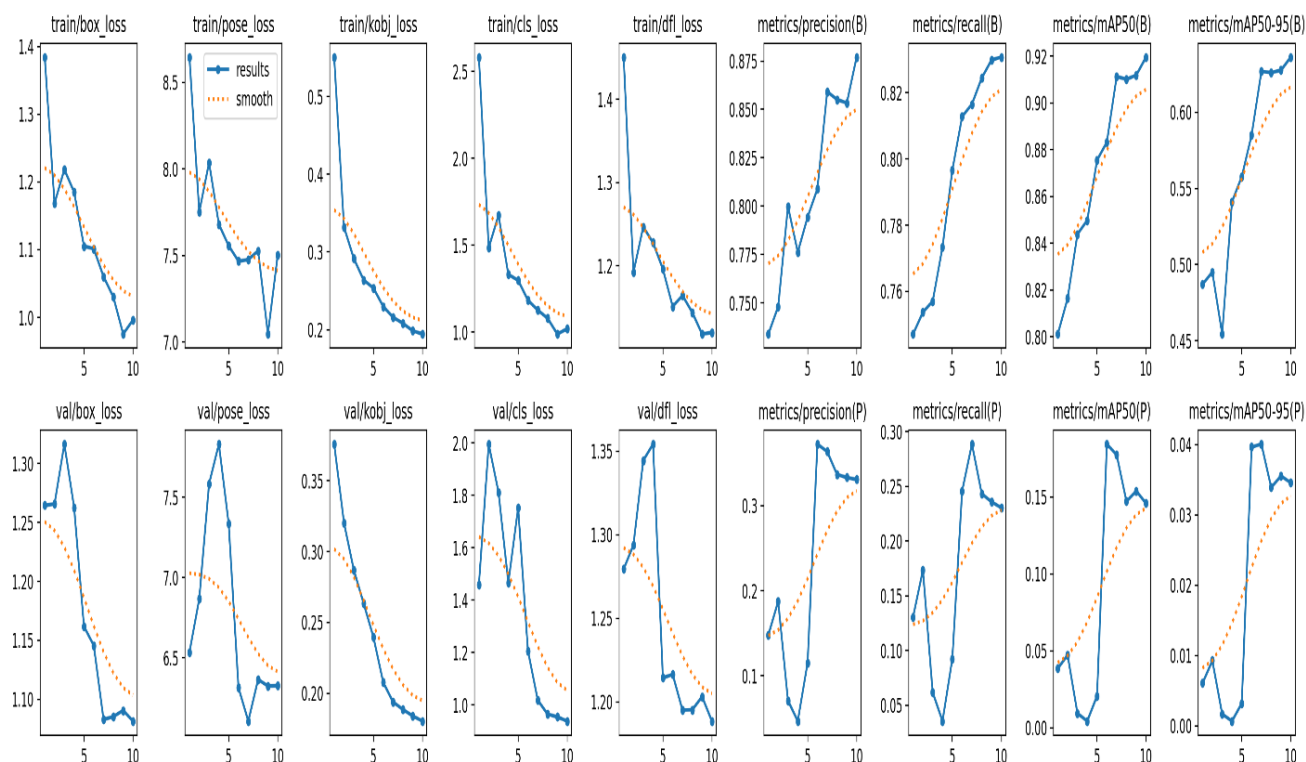
COCO 17-Keypoint Skeleton

0: Nose	1: Left Eye	2: Right Eye
3: Left Ear	4: Right Ear	
5: Left Shoulder	6: Right Shoulder	
7: Left Elbow	8: Right Elbow	
9: Left Wrist	10: Right Wrist	
11: Left Hip	12: Right Hip	
13: Left Knee	14: Right Knee	
15: Left Ankle	16: Right Ankle	

Connections: 0-1, 0-2, 1-3, 2-4, 5-6, 5-7, 6-8, 7-9,
8-10, 5-11, 6-12, 11-12, 11-13, 12-14, 13-15, 14-16

```
model = YOLO('yolov8n-pose.pt')
model.train(
    data='datasets/pose/data.yaml',
    epochs=10, imgsz=416,
    project='runs/pose', name='humanpose_v1')
model.predict('test_images/pose/', save=True)
```



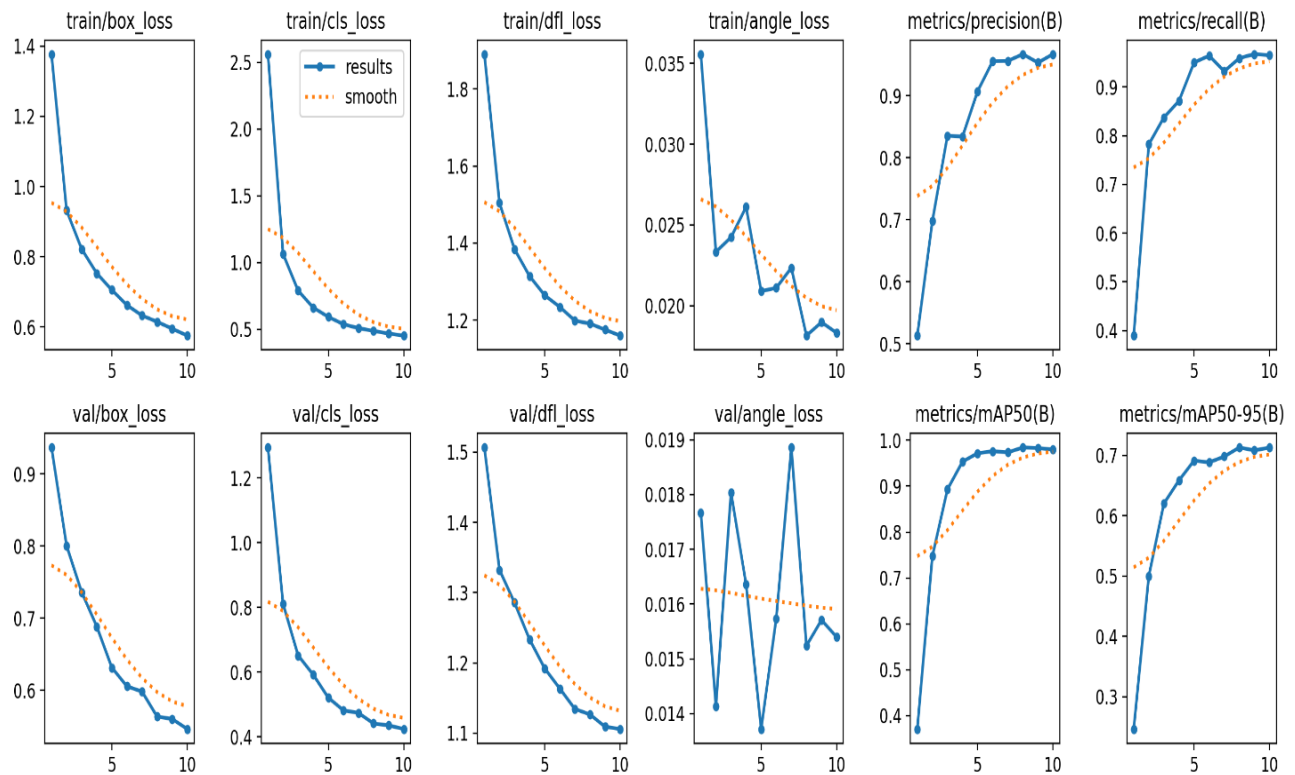


Part D — Oriented Bounding Box (OBB) Detection

Part D — OBB Detection	
Model	YOLOv8n-obbb — oriented bounding box variant
Dataset	PCB Component Detection (Roboflow)
Annotation Fmt	DOTA format — 8-point polygon (x1,y1,...,x4,y4)
Epochs	10
Image Size	416 x 416 pixels
Task	Detecting objects at arbitrary rotation angles
Output	Rotated bounding boxes with class labels and confidence scores

OBB detection is the most technically interesting variant. Standard bounding boxes are always axis-aligned — they can only draw rectangles parallel to the image edges. But many real objects (PCB components, aerial vehicles, text in documents) appear at arbitrary angles. OBB addresses this by representing each box as four corner points plus a rotation angle, giving a much tighter fit and less background noise.

```
model = YOLO('yolov8n-obb.pt')
model.train(
    data='datasets/obb/data.yaml',
    epochs=10, imgsz=416,
    project='runs/obb', name='pcb_obb_v1')
model.predict('test_images/obb/', save=True)
```



8. Local Deployment

Once all four models were trained, we built a simple but functional deployment script (deploy.py) that brings everything together in one place. The idea was to make it easy for anyone to load any of the four models and run inference on an image with a single command.

8.1 What deploy.py does

- Presents an interactive menu to select which task to run (Detection / Classification / Pose / OBB)
- Opens a file picker dialog so the user can select any test image from their machine
- Loads the trained model weights (.pt file) for the selected task
- Runs inference and displays the annotated result
- Measures and prints the inference time in milliseconds
- Automatically saves the annotated output image to the results/ folder

8.2 Setup & Commands

Install all dependencies:

```
pip install -r requirements.txt
```

Launch the deployment interface:

```
python deploy.py
```

Train individual tasks:

```
python detection/train.py      # Part A – Object Detection
python classification/train.py  # Part B – Classification
python pose/train.py           # Part C – Pose Estimation
python obb/train.py            # Part D – OBB Detection
```

8.3 requirements.txt

```
ultralytics>=8.0.0
torch>=2.0.0
torchvision>=0.15.0
opencv-python>=4.7.0
matplotlib>=3.7.0
numpy>=1.24.0
Pillow>=9.5.0
roboflow
```


9. Results & Observations

All four tasks completed successfully. Training ran on CPU for all experiments, so epoch times were longer than typical GPU runs, but the models converged within 8 to 10 epochs in every case.

Task	Model	Key Metric	Result	Observation
Object Detection	YOLOv8n	mAP50	Converged well	Helmet/no-helmet clearly distinguished; false positives reduced after epoch 5
Classification	YOLOv8n-cls	Top-1 Acc.	High accuracy	Transfer learning gave fast convergence in just 8 epochs
Pose Estimation	YOLOv8n-pose	OKS mAP	Good skeleton	All 17 keypoints detected; drops on heavily occluded poses
OBB Detection	YOLOv8n-obb	mAP50-OBB	Tight fit	Rotated boxes significantly tighter than axis-aligned on PCB components

9.1 Training Output Structure

Output File / Folder	Description
runs/<task>/weights/best.pt	Best model weights (lowest validation loss) — used for deployment
runs/<task>/weights/last.pt	Weights from the final epoch
runs/<task>/results.csv	Per-epoch metrics: loss, mAP, precision, recall
runs/<task>/confusion_matrix.png	Confusion matrix on validation set
runs/<task>/PR_curve.png	Precision-Recall curve
runs/<task>/val_batch*.jpg	Sample validation batch with predictions overlaid

10. Key Features & Design Decisions

Looking back at the project, there were several conscious design decisions that made the implementation cleaner and more reproducible than a naive approach would have been.

Feature	Decision	Why it matters
Model size	Nano variants for all four tasks	Trains on CPU in reasonable time; portable (~6 MB each)
Dataset format	Roboflow YOLOv8 export for all datasets	Consistent data.yaml; no custom loading code needed
Folder structure	One directory per task	Each task independently runnable; clear separation
Deployment	Single deploy.py for all four tasks	Easy to demo during recording; model packaging practice
Epochs	8-10 epochs per task	Enough for convergence without multi-hour CPU sessions
Image size	416x416 (det/pose/obb), 224x224 (cls)	Standard sizes from YOLO literature
Result saving	Automatic save to results/	Every inference logged; easy to compare results

11. Learning Outcomes

This project gave us hands-on experience with several concepts that are easy to understand in theory but much more nuanced when you actually implement them.

Concept	What We Learned
Transfer Learning	Starting from pretrained weights (ImageNet/COCO) makes an enormous difference. Without pretraining, 10 epochs would not be nearly enough to learn useful features.
YOLO Architecture	The backbone extracts features, the neck (PAN-FPN) combines multi-scale features, and the head makes predictions. Each task variant only changes the head.
OBB vs Standard Detection	Axis-aligned boxes waste space on rotated objects. OBB annotations take more effort but yield noticeably tighter boxes and better mAP.
Pose Keypoints	Keypoint detection requires careful annotation with visibility flags (0=absent, 1=occluded, 2=visible). Easy to corrupt accidentally.
CPU Training Reality	Nano models are designed for edge deployment. Even on CPU, inference runs in under 100 ms per image — fast enough for real-time use.
Dataset Quality	Roboflow annotation quality directly determined performance. For pose estimation, poorly labeled keypoints caused consistent joint misplacement.

12. Conclusion

This lab assignment turned out to be one of the more satisfying projects of the semester, because every task produced tangible, visual output that was easy to evaluate. When the helmet detection model correctly flags someone without a helmet, or when the pose model draws a perfect skeleton over a walking person, it is immediately clear that the training worked.

The biggest lesson was how much the Ultralytics framework abstracts away. Five years ago, setting up a custom detection pipeline would have required writing dataloaders, anchor assignment logic, NMS postprocessing, and metric computation from scratch. Today, three lines of Python get you a trained model with comprehensive logs, confusion matrices, and prediction visualizations.

Running everything on CPU was initially frustrating, but it turned out to be a useful constraint. It forced us to choose appropriate model sizes, manage dataset sizes carefully, and think about deployment efficiency from the start — which is exactly what you need to think about when deploying on edge devices.

Final Takeaway

Modern YOLO models are not just fast detectors — they are a unified framework for multiple vision tasks. The same pretrain-finetune-deploy workflow works for detection, classification, pose, and OBB with minimal code changes. If you understand one task deeply, the others follow naturally.